# Assignmnet-11.5

## Name:A.Anjali

## Hall Ticket-Name:2303A51319

## Batch-19

## Task1

### Prompt

“Generate a Python program for a Contact Manager using a list (array) that supports add, search, and delete operations. Then implement the same program using a linked list class with node objects. Include comments and compare performance of both approaches.”

```
# ================================
class ArrayContactManager:
    def __init__(self):
        self.contacts = []

    def add_contact(self, name, phone):
        self.contacts.append({"name": name, "phone": phone})
        print(f"Contact '{name}' added successfully")

    def search_contact(self, name):
        for contact in self.contacts:
            if contact["name"].lower() == name.lower():
                return contact
        return None

    def delete_contact(self, name):
        for i, contact in enumerate(self.contacts):
            if contact["name"].lower() == name.lower():
                self.contacts.pop(i)
                print(f"Contact '{name}' deleted successfully")
                return True
        print(f"Contact '{name}' not found")
        return False

    def list_contacts(self):
        return self.contacts


# ================================
# Linked List Node
# ================================
class ContactNode:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None
```

```python
# ================================
# Linked List Node
# ================================
class ContactNode:
    def __init__(self, name, phone):
        self.name = name
        self.phone = phone
        self.next = None


# ================================
# Linked List Contact Manager
# ================================
class LinkedListContactManager:
    def __init__(self):
        self.head = None

    def add_contact(self, name, phone):
        new_node = ContactNode(name, phone)
        if not self.head:
            self.head = new_node
        else:
            current = self.head
            while current.next:
                current = current.next
            current.next = new_node
        print(f"Contact '{name}' added successfully")

    def search_contact(self, name):
        current = self.head
        while current:
            if current.name.lower() == name.lower():
                return {"name": current.name, "phone": current.phone}
            current = current.next
        return None

    def delete_contact(self, name):
```

```
PS C:\Users\ashur\OneDrive\Desktop\aiac>  c:; cd 'c:\Users\ashur\OneDrive\Desktop\aiac'; & 'C:\Users\ashur\AppData\Local\Microsoft\
tensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50230' '--' 'c:\Users\ashur\OneDrive\Desktop\aiac\#A
Select Contact Manager
1. Array-Based
2. Linked List-Based
Enter choice (1 or 2): 1

===== Contact Manager =====
1. Add Contact
2. Search Contact
3. Delete Contact
4. List Contacts
5. Exit
Choose action: 1
Enter name: alice
Enter phone: 12345
Contact 'alice' added successfully

===== Contact Manager =====
```

## Analysis

The prompt clearly specifies two implementations, helping the AI generate structured solutions.

By mentioning operations explicitly, the output becomes task-focused and avoids unnecessary code.

The comparison requirement encourages the AI to include conceptual explanations, not just syntax.

AI assistance speeds up writing search/delete methods, which often involve loops or pointer updates.

This task shows how arrays are faster for searching but linked lists are better for insertions/deletions.

It demonstrates how AI can support both coding and conceptual understanding.

# Task2

## Prompt

“Create a Python program implementing a queue for book requests using FIFO. Extend it to a priority queue where faculty requests have higher priority than students. Implement enqueue() and dequeue() methods and simulate sample requests.”

from collections import deque.py > ...

```python
from collections import deque
import heapq


# ==============================
# Book Request Class
# ==============================
class BookRequest:
    def __init__(self, request_id, requester_name, requester_type, book_title):
        self.request_id = request_id
        self.requester_name = requester_name
        self.requester_type = requester_type.lower()  # 'student' or 'faculty'
        self.book_title = book_title

    def __repr__(self):
        return (f"BookRequest(ID: {self.request_id}, "
                f"{self.requester_name} ({self.requester_type}), "
                f"Book: {self.book_title})")


# ==============================
# Simple Queue (FIFO)
# ==============================
class SimpleQueue:
    def __init__(self):
        self.queue = deque()

    def enqueue(self, request):
        self.queue.append(request)
        print(f" Enqueued: {request}")

    def dequeue(self):
        if self.queue:
            request = self.queue.popleft()
            print(f" Dequeued: {request}")
            return request
        else:
```

```
ython3.12.exe' 'c:\Users\ashur\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '53328' '--' 'c:\User
hur\OneDrive\Desktop\aiac\from collections import deque.py'
============================================================
 SRU LIBRARY BOOK REQUEST SYSTEM
============================================================
1. Simple Queue (FIFO)
2. Priority Queue (Faculty First)
Select system (1 or 2): 2

1. Add Book Request
2. Process Next Request
3. View Pending Requests
4. Exit
Choose option: 1
Enter requester name: alice
Enter requester type (student/faculty): student
Enter book title: 101
 Enqueued (Priority): BookRequest(ID: 1, alice (student), Book: 101)

1. Add Book Request
2. Process Next Request
3. View Pending Requests
4. Exit
Choose option: 2
 Dequeued (Priority): BookRequest(ID: 1, alice (student), Book: 101)
```

## Analysis

This prompt helps the AI distinguish between normal queues and priority queues.

Specifying FIFO ensures the generated queue behaves correctly.

The priority condition ensures the AI includes sorting, heap usage, or priority values.

Simulation instructions make the output practical and testable.

AI-generated methods reduce manual coding time for queue logic.

The task highlights real-world scheduling problems solved using queues.

# Task3

## Prompt

“Write a Python program implementing a stack to manage IT support tickets. Include push, pop, and peek operations. Add functions to check if the stack is empty or full and simulate resolving five tickets.”

```python
class Stack:
    def __init__(self, max_size=100):
        self.items = []
        self.max_size = max_size

    def push(self, ticket):
        if self.is_full():
            print(" Stack is full! Cannot add more tickets.")
            return False
        self.items.append(ticket)
        print(f" Ticket added: {ticket}")
        return True

    def pop(self):
        if self.is_empty():
            print(" No tickets to resolve!")
            return None
        ticket = self.items.pop()
        print(f" Ticket resolved: {ticket}")
        return ticket

    def peek(self):
        if self.is_empty():
            print(" No tickets available.")
            return None
        return self.items[-1]

    def is_empty(self):
        return len(self.items) == 0

    def is_full(self):
        return len(self.items) >= self.max_size

    def size(self):
        return len(self.items)

    def display(self):
```

```
PS C:\Users\ashur\OneDrive\Desktop\aiac>  c:; cd 'c:\Users\ashur\OneDrive\Desktop\aiac'; & 'C:\Users\ashur\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\ashur\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '50755' '--' 'c:\Users\ashur\OneDrive\Desktop\aiac\11.5.py'
==============================================
 IT HELP DESK TICKET SYSTEM (STACK - LIFO)
==============================================
Enter maximum stack size: 1

1. Add Ticket
2. Resolve Ticket
3. View Next Ticket
4. Display All Tickets
5. Check Stack Size
6. Check if Empty
7. Check if Full
8. Exit
Choose option: 1
Enter issue description: password reset
 Ticket added: Ticket #001: password reset

1. Add Ticket
2. Resolve Ticket
3. View Next Ticket
4. Display All Tickets
5. Check Stack Size
6. Check if Empty
7. Check if Full
8. Exit
Choose option: 1
Enter issue description: 2
 Stack is full! Cannot add more tickets.

1. Add Ticket
2. Resolve Ticket
3. View Next Ticket
4. Display All Tickets
5. Check Stack Size
```

## Analysis

The prompt clearly states LIFO behavior, guiding AI toward stack logic.

Mentioning required operations ensures a complete implementation.

Adding simulation forces the AI to include test cases and outputs.

AI suggestions often include built-in list usage or class-based stacks.

This task reinforces the difference between stacks and queues.

It also demonstrates how AI can suggest additional validation methods.

# Task4

## Prompt

“Generate a Python implementation of a hash table class with insert, search, and delete methods. Use chaining for collision handling and include comments explaining how hashing and collisions work.”

```python
class HashTable:
    def __init__(self, size=10):
        """Initialize hash table with given size."""
        self.size = size
        self.table = [[] for _ in range(size)]

    def _hash(self, key):
        """Generate hash value for a given key."""
        return hash(key) % self.size

    def insert(self, key, value):
        """Insert a key-value pair into the hash table."""
        index = self._hash(key)
        bucket = self.table[index]

        # Update if key already exists
        for i, (k, v) in enumerate(bucket):
            if k == key:
                bucket[i] = (key, value)
                return

        # Insert new key-value pair
        bucket.append((key, value))

    def search(self, key):
        """Search for a value by key. Returns value or None."""
        index = self._hash(key)
        bucket = self.table[index]

        # Linear search in bucket (chaining)
        for k, v in bucket:
            if k == key:
                return v

        return None

    def delete(self, key):
```

```
PS C:\Users\ashur\OneDrive\Desktop\aiac> & 'C:\Users\ashur\AppData\Local\Microsoft\WindowsApps\python3.12.exe' 'c:\Users\ashur\.vscode\extensio
ns\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '64914' '--' 'c:\Users\ashur\OneDrive\Desktop\aiac\11.3.py'
Alice
None
PS C:\Users\ashur\OneDrive\Desktop\aiac>
```

## Analysis

This prompt ensures the AI includes collision handling instead of a basic dictionary.

By specifying chaining, the AI usually uses lists at each index.

Comment requirement improves readability and educational value.

AI-generated hash functions may vary, showing different implementation ideas.

The task helps students understand indexing and time complexity.

It shows how AI can assist in building more advanced data structures.

# Task5

## Prompt

“Suggest suitable data structures for attendance tracking, event registration, library borrowing, bus scheduling, and cafeteria order queue. Justify each choice in 2–3 sentences and implement one feature in Python.”

```python
from datetime import datetime
from collections import defaultdict

class AttendanceTracker:
    """
    Using Hash Map: O(1) lookup and insertion for quick attendance marking.
    Justification: Enables fast student lookup by ID, efficient date-based
    attendance records, and quick mark/retrieve operations in real-time.
    """

    def __init__(self):
        self.attendance = defaultdict(lambda: {})

    def mark_attendance(self, student_id, date=None):
        if date is None:
            date = datetime.now().strftime("%Y-%m-%d")
        self.attendance[student_id][date] = "Present"

    def get_attendance(self, student_id):
        return self.attendance.get(student_id, {})

    def attendance_percentage(self, student_id, total_days):
        records = self.attendance.get(student_id, {})
        present_days = len(records)
        return (present_days / total_days * 100) if total_days > 0 else 0

# Example usage
tracker = AttendanceTracker()
tracker.mark_attendance("STU001")
tracker.mark_attendance("STU002")
print(tracker.get_attendance("STU001"))
print(f"Attendance: {tracker.attendance_percentage('STU001', 30):.1f}%")
```

```
PS C:\Users\ashur\OneDrive\Desktop\aiac>  c:; cd 'c:\Users\ashur\OneDrive\Desktop\aiac'; & 'C:\Users\ashur\AppData\Local\Microsoft\WindowsApps\p
ython3.12.exe' 'c:\Users\ashur\.vscode\extensions\ms-python.debugpy-2025.18.0-win32-x64\bundled\libs\debugpy\launcher' '54214' '--' 'c:\Users\as
hur\OneDrive\Desktop\aiac\11.1.py'
{'2026-02-25': 'Present'}
Attendance: 3.3%
PS C:\Users\ashur\OneDrive\Desktop\aiac>
```

## Analysis

This prompt focuses on design thinking rather than only coding.

It encourages the AI to explain why a data structure fits a problem.

Justification requirements improve conceptual clarity.

Implementation of one feature keeps the solution practical.

AI assistance helps in selecting efficient structures like queues, hash maps, or trees.

This task shows how AI supports decision-making in software design.